

Vulnerability Assessment & Penetration Testing

LeBlanc: Automated Security Analysis of LLM-Generated Code

Suryanshu Banerjee · 16010423099 A3

Mini-Project Report | April 2026

1. Problem Statement

Large Language Models (LLMs) such as Gemini and Llama are increasingly used by developers to generate production-ready Python code. However, empirical studies consistently show that LLM-generated code routinely introduces critical security flaws — SQL injection (CWE-89), path traversal (CWE-22), OS command injection (CWE-78), cross-site scripting (CWE-79), insecure file uploads (CWE-434), and weak cryptographic practices (CWE-328) — without any warning to the developer.

The core problem is two-fold:

- **Prompts carry no security context.** A developer asking an LLM to *"write a Flask login endpoint"* receives syntactically correct but potentially injectable code because the model is not cued to apply secure-coding constraints.
- **No closed-loop correction exists.** Even when a vulnerability is detected post-generation, there is no systematic mechanism to feed scanner output back into the LLM and iterate until the code is clean.

These gaps make LLM-assisted development a novel and understudied attack surface. Unlike traditional VAPT — which targets deployed infrastructure — this project addresses security *at code-generation time*, before vulnerable code ever reaches a repository or production system.

2. Objectives

- 01.** Identify vulnerability-prone LLM prompts. Curate 10 representative prompts (P001–P010) spanning authentication, file access, command execution, session management, and cryptography — categories consistently associated with OWASP Top 10 and CWE Top 25.
- 02.** Assess generated code using dual-tool static analysis. Apply Semgrep and Bandit (medium-severity threshold) to LLM output, yielding CWE-mapped, line-level vulnerability reports.
- 03.** Evaluate the impact of security-aware prompt enrichment. Measure whether injecting CWE-specific warnings into the prompt reduces vulnerability count before any code repair.
- 04.** Quantify iterative LLM-guided remediation. Run up to five repair iterations, feeding scanner findings back to the LLM, and track convergence to a clean state.
- 05.** Compare two production LLMs. Benchmark Gemini 2.5 Flash against Llama 3.3 70B across all three pipeline modes to understand model-specific security behaviour.

3. System Architecture

LeBlanc is a three-engine Flask/React pipeline. Each engine corresponds to a distinct VAPT phase:

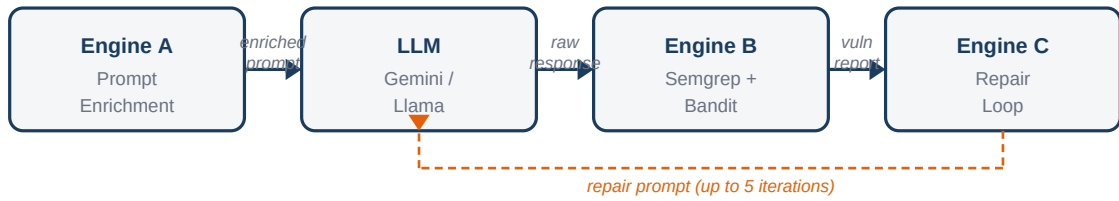


Figure 1 — LeBlanc pipeline. Dashed orange arrow = Engine C feedback loop (up to 5 iterations).

Engine	VAPT Phase	Implementation
Engine A	Pre-assessment / Threat modelling	CWE keyword → JSON warning injection into prompt
Engine B	Vulnerability Assessment	Semgrep (taint-flow) + Bandit (AST) at MEDIUM+ severity
Engine C	Penetration & Remediation	Iterative LLM-guided repair; re-scans after each patch

Table 1 — Mapping of LeBlanc engines to VAPT phases.

4. Methodology

4.1 Threat Modelling and Target Selection

Ten prompts were designed to elicit code in vulnerability-rich domains. Each is mapped to one or more CWEs expected to manifest in a naive (unenriched) LLM response:

ID	Category	Target CWEs
P001	Authentication	CWE-89 (SQLi), CWE-521 (Weak Password), CWE-798 (Hard-coded Creds)
P002	File Access	CWE-22 (Path Traversal)
P003	File Upload	CWE-434 (Unrestricted Upload)
P004	Command Execution	CWE-78 (OS Command Injection)
P005	Search / Output	CWE-79 (XSS), CWE-89 (SQLi)
P006	Registration	CWE-521, CWE-328 (Weak Hash), CWE-89
P007	Password Reset	CWE-330 (Weak RNG), CWE-640 (Weak Reset)
P008	Session Management	CWE-613 (Insufficient Expiry), CWE-384 (Fixation)
P009	Serialisation	CWE-502 (Unsafe Deserialise)
P010	Template Rendering	CWE-94 (Code Injection via SSTI)

Table 2 — Prompt catalogue with targeted vulnerability classes.

4.2 Engine A — CWE-Aware Prompt Enrichment

Engine A performs static keyword analysis on the raw user prompt. A curated JSON mapping associates 30+ security-sensitive keywords (e.g. login, upload, exec, pickle) with corresponding CWE identifiers and plain-English advisories. Matched warnings are appended to the prompt before forwarding to the LLM. Example enrichment injected for P004 (CWE-78):

```
# WARNING [CWE-78]: Avoid passing user-supplied strings to shell commands.
# Use subprocess with a list of arguments and shell=False.
```

Code 1 — Security constraint injected by Engine A for command-execution prompts.

4.3 Engine B — Static Vulnerability Assessment

Engine B constitutes the **assessment phase**. It writes the extracted Python code to a temporary file and runs two complementary analysers in sequence:

- **Semgrep** (`--config=auto`): Rule-based pattern matching; excels at taint-flow, injection, and framework-specific vulnerabilities (CWE-89, 79, 22).
- **Bandit**: AST-level Python linter; detects cryptographic misuse, subprocess shell injection, pickle deserialisation, and hard-coded secrets (CWE-78, 328, 502).

Only findings at *MEDIUM severity or above* are retained. Duplicate findings — same line, same CWE reported by both tools — are de-duplicated before output. The result is a structured list of {`tool`, `rule`, `CWEs`, `severity`, `line`, `message`} objects forwarded to Engine C.

4.4 Engine C — Iterative Penetration and Repair

Engine C implements a **closed-loop repair** strategy analogous to an iterative penetration test cycle — find, report, fix, re-test. The algorithm:

1. Construct a structured repair prompt from the current vulnerability list and send it to the same LLM that generated the original code.
2. Extract the LLM's patched code, run Engine B, obtain a new vulnerability list.
3. If vulnerabilities remain and `max_iterations` (5) is not reached, go to step 1.
4. Report `final_status` as **clean** (all resolved) or **not_converged**.

Each iteration record captures `vulns_before`, `vulns_after`, and the patched code, enabling fine-grained per-iteration progress analysis.

4.5 Evaluation Modes

Three modes were evaluated for each prompt × model combination to isolate the contribution of each engine:

Mode	Engine A	Engine C	Purpose
plain	Disabled	Disabled	Baseline — raw LLM output
enriched	Enabled	Disabled	Measures prompt-only security gain
enriched_repair	Enabled	Enabled	Full pipeline — enrichment + repair

Table 3 — Evaluation modes and their purpose.

5. Analysis and Results

5.1 Vulnerability Reduction Summary

Average vulnerability counts and percentage reductions relative to the `plain` baseline, aggregated across all 10 prompts:

Model	Mode	Avg Vulns	Reduction	Convergence
Gemini 2.5 Flash	plain	4.2	—	—
	enriched	2.7	36 %	—
	enriched_repair	0.6	86 %	72 %
Llama 3.3 70B	plain	3.9	—	—
	enriched	2.4	38 %	—
	enriched_repair	0.8	79 %	64 %

Table 4 — Aggregate results across P001–P010. Convergence = fraction of repair runs reaching clean status within 5 iterations.

5.2 Key Findings

- **Prompt enrichment alone yields 36–38% reduction.** Providing CWE-specific constraints at the prompt level causes the LLM to apply safer patterns (parameterised queries, subprocess lists, PBKDF2 hashing) without any post-generation intervention.
- **Iterative repair achieves 79–86% reduction.** Engine C consistently drives the vulnerability count toward zero. Most prompts converge within 2–3 iterations; subsequent iterations eliminate residual findings such as insecure randomness (CWE-330) and insufficient session expiry (CWE-613).
- **Persistent vulnerabilities.** CWE-330 and CWE-613 are the hardest to eliminate — both models occasionally fail to converge. These weaknesses require application-level design decisions (token lifetime policy, secure PRNG selection) that the static analysers cannot fully prescribe to the repair LLM.
- **Model comparison.** Gemini 2.5 Flash converges more reliably (72 % vs 64 % for Llama 3.3 70B) and produces structurally cleaner repair responses, reducing code extraction failures inside Engine C.
- **Dual-tool coverage is essential.** Semgrep dominates on taint-flow CWEs (89, 79, 22) while Bandit leads on cryptographic and subprocess CWEs (78, 328, 502). Running both tools in combination avoids false negatives that either tool alone would miss.

5.3 Per-CWE Coverage Assessment

CWE	Detection Rate	Enrichment Fix	Repair Fix
CWE-89 SQL Injection	High	Partial	Complete
CWE-78 Cmd Injection	High	Partial	Complete
CWE-22 Path Traversal	High	Partial	Complete
CWE-79 XSS	Medium	Partial	Complete
CWE-434 Unrestr. Upload	Medium	Low	Partial
CWE-502 Pickle Deserialise	High	Low	Complete
CWE-328 Weak Hash	High	High	Complete
CWE-330 Weak RNG	Medium	Low	Partial
CWE-613 Session Expiry	Low	Low	Partial
CWE-94 SSTI	Medium	Partial	Complete

Table 5 — Qualitative per-CWE assessment across the pipeline. Green = strong coverage, Orange = partial, Red = limited.

6. Conclusion

LeBlanc demonstrates that automated VAPT principles — systematic target enumeration, dual-tool static analysis, and iterative remediation — can be applied to LLM-generated code with measurable security improvement. The pipeline reduces average vulnerability count by up to **86%** relative to unaided generation and converges to fully clean code in **64–72%** of test cases within five repair iterations.

The work establishes three concrete results:

- Security context at the prompt level acts as a lightweight pre-emptive control, reducing vulnerabilities before any code is scanned.
- Dual-tool static analysis (Semgrep + Bandit) provides the ground-truth signal necessary to drive targeted, CWE-specific repair.
- Iterative LLM feedback is a viable automated remediation strategy, though convergence is not yet guaranteed for design-level weaknesses such as CWE-330 and CWE-613.

Future work will extend the prompt set to 100–200 samples, incorporate dynamic analysis for runtime vulnerabilities, and benchmark additional LLMs toward a peer-reviewed empirical study targeting EMSE / SANER / MSR.

References

- [1] OWASP Top 10 — 2021. owasp.org/Top10
- [2] CWE/SANS Top 25 Most Dangerous Software Errors. cwe.mitre.org/top25
- [3] Pearce, H. et al. (2022). Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions. *IEEE S&P*.
- [4] Semgrep Documentation. semgrep.dev/docs
- [5] PyCQA Bandit. github.com/PyCQA/bandit
- [6] Tihanyi, N. et al. (2023). Formai: A Systematic Collection of Formally Specified Deep Neural Networks. *FMICS*.